

Лабораторная работа № 7

Дискретно-событийное моделирование

Кира Андреевна Нечаева

В данном отчёте представлены результаты выполнения лабораторной работы по дискретно-событийному моделированию.

Рассмотрены модель массового обслуживания $M/M/c$ и модель резервирования и ремонта Росса.

Проведены вычислительные эксперименты, построены графики и выполнено сравнение имитационного и аналитического решений.

Содержание

1	Цель работы	7
2	Подготовка рабочего окружения	8
3	Модель М/М/с	10
3.1	Постановка	10
3.2	Реализация	10
4	Базовая модель Росса	13
4.1	Постановка	13
4.2	Базовый прогон	13
5	Расширенная модель Росса	15
6	Мониторинг системы	17
6.1	Число исправных машин	18
6.2	Очередь на ремонт	19
6.3	Загрузка ремонтников	20
7	Сравнение с аналитическим решением	21
8	Литературное программирование	24
9	Проверка воспроизводимости	25
10	Структура проекта	26
11	Контроль версий	28
12	Выводы	29
	Приложение А. Документация модели М/М/с	30
13	Модель М/М/с	31
13.1	Параметры моделирования	32
13.2	Параметры очереди	32
13.3	Поведение заявки	33
13.4	Запуск моделирования	35

13.5	Результаты	37
13.6	График событий	39
13.7	Время ожидания	40
Приложение Б. Документация базовой модели Росса		42
14	Модель Росса	43
14.1	Поведение машины	45
14.2	Начальное состояние	47
14.3	Один прогон	49
14.4	Несколько повторов	50
Приложение В. Документация параметрической модели Росса		52
15	Расширение модели Росса	53
15.1	Параметры из исходной модели	54
15.2	Мониторинг	54
15.3	Поведение машины	56
15.4	Начальное состояние	61
15.5	Один параметрический прогон	63
15.6	Среднее значение показателя во времени	65
15.7	Аналитическое среднее время до падения	67
15.8	Набор параметров	70
15.9	Серия экспериментов	71
15.10	Таблица результатов	76
15.11	График числа исправных машин	77
15.12	График очереди на ремонт	78
15.13	Загрузка ремонтников	79
15.14	Имитационное и аналитическое время падения	80

Список иллюстраций

2.1	Начальное состояние Git и удалённые репозитории	8
2.2	Проверка рабочего окружения	9
3.1	Результаты моделирования М/М/с	11
3.2	Времена событий заявок в М/М/с	12
3.3	Время ожидания заявок	12
4.1	Базовые прогоны модели Росса	14
5.1	Выполнение расширенного параметрического эксперимента	16
6.1	Созданные таблицы и графики	17
6.2	Изменение числа исправных машин	18
6.3	Длина очереди на ремонт	19
6.4	Загрузка ремонтников	20
7.1	Таблица сравнения имитационного и аналитического решений . . .	22
7.2	Имитационное и аналитическое время до падения	23
8.1	Сгенерированные Quarto-документы и Jupyter Notebook	24
9.1	Выполнение Jupyter Notebook	25
10.1	Итоговая структура проекта	27
11.1	Финальное состояние Git	28

Список таблиц

1 Цель работы

Цель лабораторной работы — изучить дискретно-событийное моделирование на примере системы массового обслуживания $M/M/c$ и модели резервирования и ремонта Росса.

В первой части моделируется поток заявок, поступающих в систему с несколькими параллельными каналами обслуживания.

Во второй части моделируется система машин, резервов и ремонтных устройств. Дополнительно исследуется влияние числа машин и ремонтников, проводится мониторинг очереди на ремонт и загрузки ремонтников, а результаты имитационного моделирования сравниваются с аналитическим решением.

2 Подготовка рабочего окружения

Работа выполнялась в Ubuntu через WSL в ветке develop общего Git-репозитория.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git pull --rebase origin develop
From ssh://gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling
 * branch          develop      -> FETCH_HEAD
Already up to date.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git remote -v
github  git@github.com:kanetschaeva/2026-1-study-simulation-modeling.git (fetch)
github  git@github.com:kanetschaeva/2026-1-study-simulation-modeling.git (push)
origin  ssh://git@gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling.git (fetch)
origin  ssh://git@gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling.git (push)
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ |
```

Рисунок 2.1: Начальное состояние Git и удалённые репозитории

Для реализации дискретно-событийных моделей использовались `ConcurrentSim`, `ResumableFunctions`, `Distributions` и `StableRNGs`.

Дополнительно использовались пакеты для работы с таблицами, графиками, Jupyter Notebook и Quarto.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ julia --project=. scripts/test_setup.jl
Проверка окружения:

✓ DrWatson
✓ StableRNGs
✓ Distributions
✓ ConcurrentSim
✓ ResumableFunctions
✓ DataFrames
✓ CSV
✓ Plots
✓ Literate
✓ IJulia
✓ Quarto
✓ Random
✓ Statistics
✓ LinearAlgebra

Окружение готово.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ |
```

Рисунок 2.2: Проверка рабочего окружения

3 Модель M/M/c

3.1 Постановка

Модель M/M/c описывает систему массового обслуживания с пуассоновским входящим потоком, экспоненциальным временем обслуживания и несколькими параллельными каналами.

В базовом эксперименте использовались:

- 10 заявок;
- 2 канала обслуживания;
- интенсивность поступления $\lambda = 0.9$;
- параметр обслуживания $\mu = 1/2$;
- фиксированное зерно генератора случайных чисел.

3.2 Реализация

Каждая заявка создаётся как отдельный процесс.

Сначала она ожидает момента своего поступления, затем запрашивает ресурс Resource. Если канал свободен, начинается обслуживание. После случайного времени обслуживания ресурс освобождается.

Дополнительно сохранялись:

- время поступления;
- время начала обслуживания;

- время выхода;
- время ожидания;
- полное время нахождения в системе.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ julia --project=. scripts/mmc.jl

Customer 1 arrived: 0.14518451436852475
Customer 1 entered service: 0.14518451436852475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085

Среднее время ожидания: 4.856795873206697
Среднее время в системе: 8.410492119082418

Сохранено:
data/mmc_results.csv
plots/mmc_timeline.png
plots/mmc_waiting_time.png
```

Рисунок 3.1: Результаты моделирования М/М/с

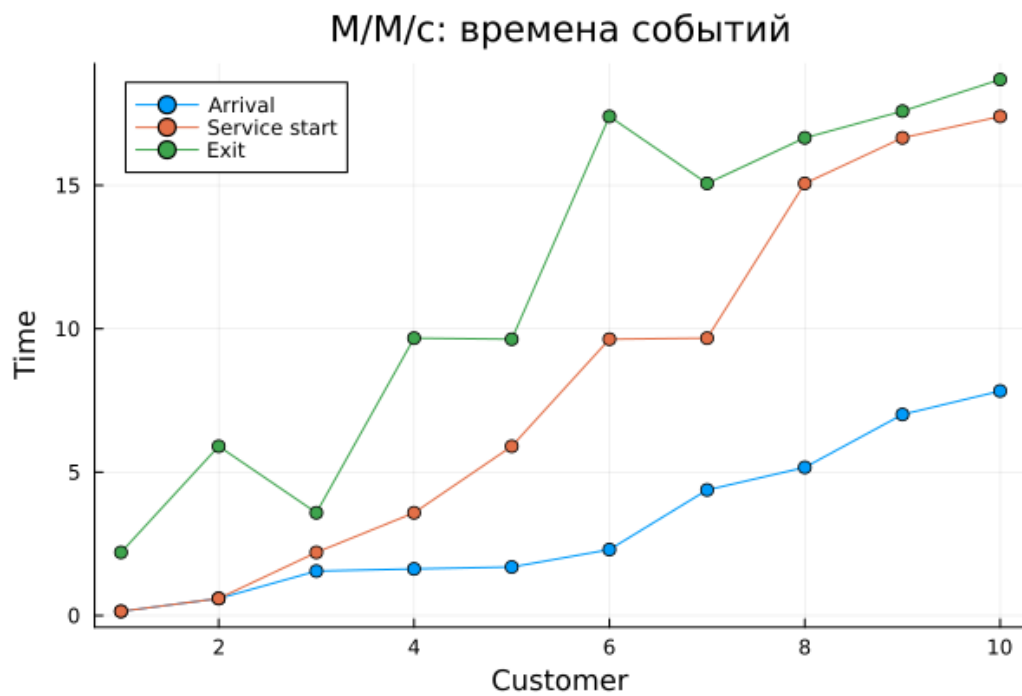


Рисунок 3.2: Времена событий заявок в М/М/с

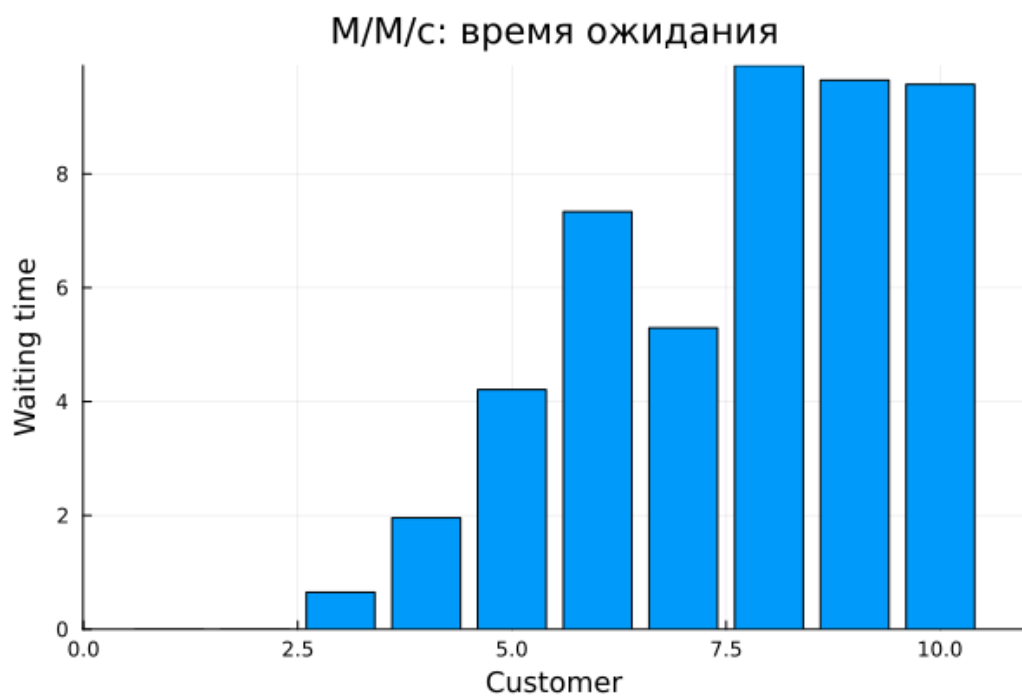


Рисунок 3.3: Время ожидания заявок

4 Базовая модель Росса

4.1 Постановка

В модели рассматриваются работающие машины, резервные машины и ремонтное устройство.

В базовом варианте используются:

- 10 основных машин;
- 3 резервные машины;
- 1 ремонтник;
- среднее время до отказа одной машины 100 часов;
- среднее время ремонта 1 час;
- 5 повторных прогонов.

При отказе работающей машины предпринимается попытка взять исправную машину из резерва.

Если резерв пуст, система считается отказавшей и моделирование завершается.

4.2 Базовый прогон

Сначала была выполнена исходная реализация модели Росса.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ julia --project=. scripts/ross.jl
At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$
```

Рисунок 4.1: Базовые прогоны модели Росса

По пяти прогонам вычисляется среднее время до падения системы.

5 Расширенная модель Росса

По заданию модель была дополнена возможностью изменять число основных машин и количество ремонтников.

Использовались:

- $N = 5, 10, 15$;
- 3 резервные машины;
- 1 или 2 ремонтника;
- несколько повторных запусков для каждого сценария.

Для каждого варианта рассчитывались:

- среднее время до падения;
- средняя длина очереди на ремонт;
- загрузка ремонтников;
- аналитическое среднее время до падения;
- относительное отличие имитационного и аналитического результата.

```

kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ julia --project=. scripts/ross__parameter.jl
N=5, repairers=1 | simulation=97701.92 | analytic=177280.0 | queue=0.0031 | utilization=0.05
N=5, repairers=2 | simulation=775098.954 | analytic=690080.0 | queue=0.0 | utilization=0.0249
N=10, repairers=1 | simulation=7185.209 | analytic=12340.0 | queue=0.0151 | utilization=0.103
N=10, repairers=2 | simulation=31982.228 | analytic=46540.0 | queue=0.0003 | utilization=0.0502
N=15, repairers=1 | simulation=1954.355 | analytic=2727.901 | queue=0.0233 | utilization=0.1383
N=15, repairers=2 | simulation=10539.718 | analytic=9927.901 | queue=0.0023 | utilization=0.0809

Сводная таблица:
6x8 DataFrame
Row | N      S      repairers mean_crash_time analytic_crash_time relative_error
    | Int64 Int64 Int64      Float64      Float64      Float64
4   |-----|-----|-----|-----|-----|-----|
1 | 5      3      1      97701.9      177280.0
0.448884 | 0.00306157 | 0.050042
2 | 5      3      2      775099.0      690080.0
0.123202 | 4.08592e-5 | 0.0249472
3 | 10     3      1      7185.21      12340.0
0.41773 | 0.0151299 | 0.102958
4 | 10     3      2      31982.2      46540.0
0.312801 | 0.00028157 | 0.0502449
5 | 15     3      1      1954.36      2727.9
0.283568 | 0.0232635 | 0.138309
6 | 15     3      2      10539.7      9927.9
0.061626 | 0.00225994 | 0.0808827

Сохранено:
data/ross_parameter_summary.csv
plots/ross_healthy.png
plots/ross_queue.png
plots/ross_utilization.png

```

Рисунок 5.1: Выполнение расширенного параметрического эксперимента

6 Мониторинг системы

Во время моделирования сохранялись состояния системы во времени.

Отдельно отслеживались:

- количество исправных машин;
- длина очереди на ремонт;
- количество занятых ремонтников.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ echo "=== DATA ==="
find data -maxdepth 1 -type f | sort
=== DATA ===
data/mmc_results.csv
data/ross_parameter_summary.csv
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ echo "=== PLOTS ==="
find plots -maxdepth 1 -type f | sort
=== PLOTS ===
plots/mmc_timeline.png
plots/mmc_waiting_time.png
plots/ross_crash_time.png
plots/ross_healthy.png
plots/ross_queue.png
plots/ross_utilization.png
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ |
```

Рисунок 6.1: Созданные таблицы и графики

6.1 Число исправных машин

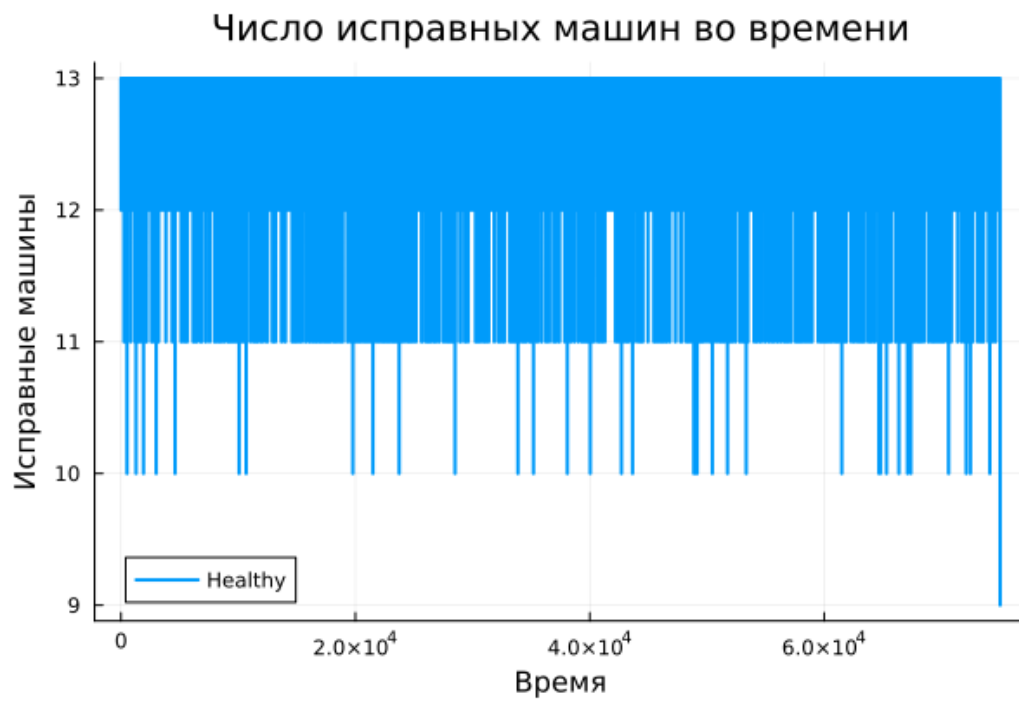


Рисунок 6.2: Изменение числа исправных машин

График показывает последовательность отказов и ремонтов до момента падения системы.

6.2 Очередь на ремонт

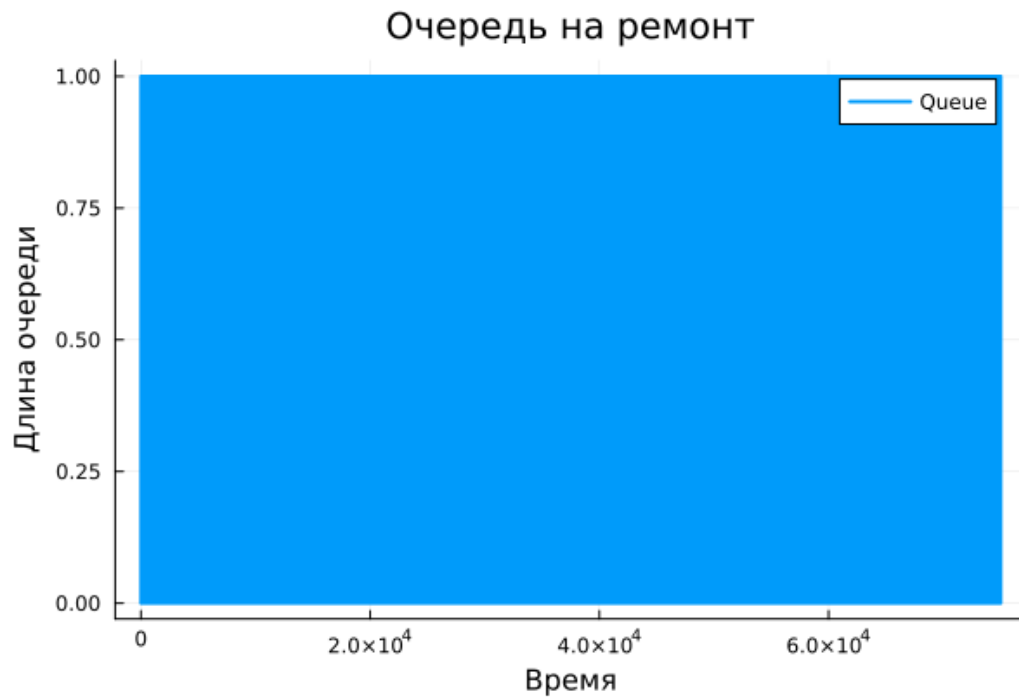


Рисунок 6.3: Длина очереди на ремонт

Очередь возникает, когда отказавших машин временно становится больше, чем ремонтники могут обслужить одновременно.

6.3 Загрузка ремонтников

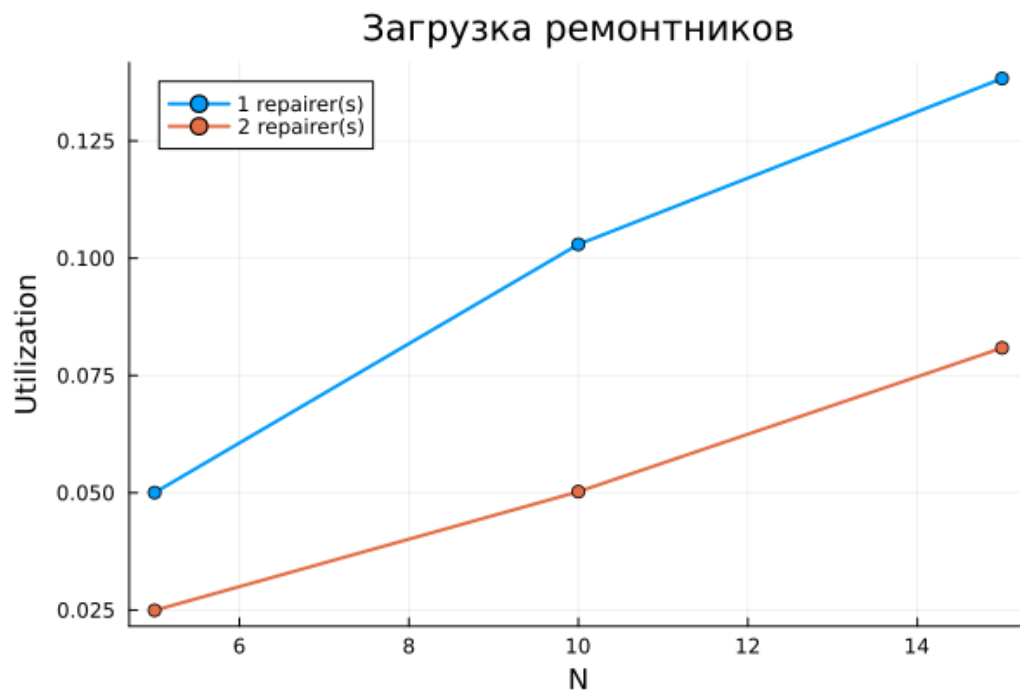


Рисунок 6.4: Загрузка ремонтников

Сравнение выполняется для одного и двух ремонтников и разных значений N.

7 Сравнение с аналитическим решением

Для аналитического расчёта использовалось марковское представление системы.

Состояние определяется количеством исправных машин.

При отказе число исправных машин уменьшается, при завершении ремонта — увеличивается.

Система линейных уравнений для средних времён достижения состояния отказа формируется и решается программно.

```

kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ julia --project=. -e '
using CSV, DataFrames
df = CSV.read("data/ross_parameter_summary.csv", DataFrame)
select!(df, :N, :repairers, :mean_crash_time, :analytic_crash_time, :relative_error)
show(df; allrows=true, allcols=true)
println()
'
6×5 DataFrame
Row | N      repairers  mean_crash_time  analytic_crash_time  relative_error
   | Int64  Int64      Float64          Float64          Float64
---+-----
1 | 5      1          97701.9         177280.0          0.44888
4 | 5      2          775099.0        690080.0          0.12320
2 | 10     1           7185.21         12340.0           0.41773
4 | 10     2          31982.2         46540.0           0.31280
1 | 15     1          1954.36         2727.9            0.28356
8 | 15     2          10539.7         9927.9            0.06162
6
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$

```

Рисунок 7.1: Таблица сравнения имитационного и аналитического решений

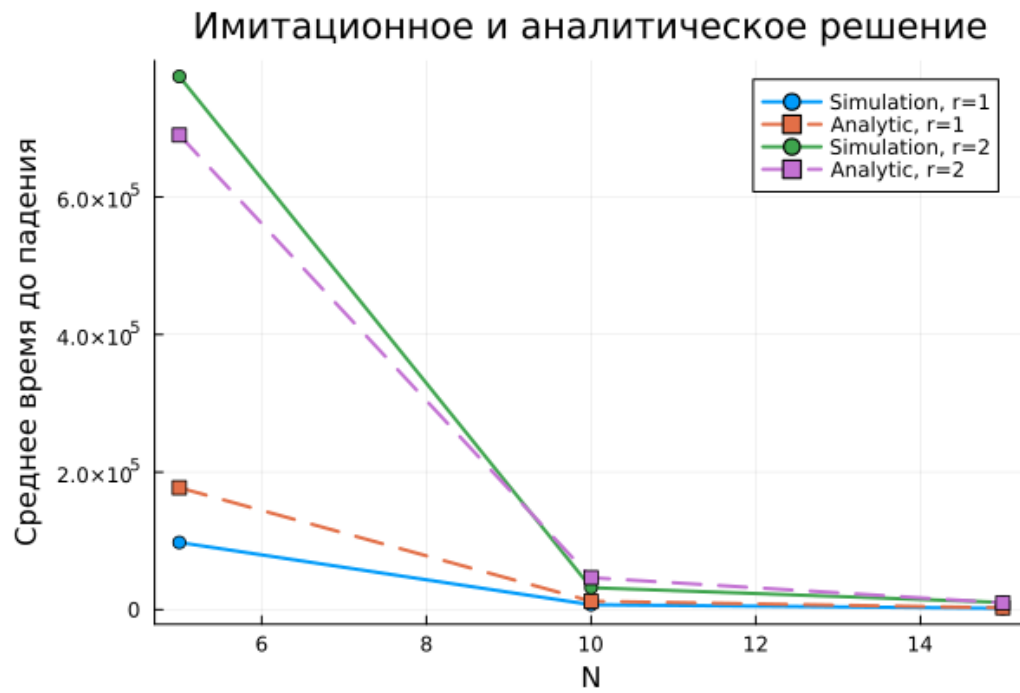


Рисунок 7.2: Имитационное и аналитическое время до падения

Различия между отдельными значениями связаны со случайным характером имитационных запусков и ограниченным числом повторений.

8 Литературное программирование

Программы были оформлены в литературном стиле.

Из исходных файлов автоматически создавались:

- чистый Julia-код;
- Jupyter Notebook;
- документация Quarto.

```
Готово! Все файлы созданы.  
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ echo "=== QUARTO ==="  
find markdown -type f -name "*.qmd" | sort  
=== QUARTO ===  
markdown/mmc/mmc.qmd  
markdown/ross/ross.qmd  
markdown/ross__param/ross__param.qmd  
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ echo "=== JUPYTER ==="  
find notebooks -type f -name "*.ipynb" | sort  
=== JUPYTER ===  
notebooks/mmc/mmc.ipynb  
notebooks/ross/ross.ipynb  
notebooks/ross__param/ross__param.ipynb  
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ |
```

Рисунок 8.1: Сгенерированные Quarto-документы и Jupyter Notebook

9 Проверка воспроизводимости

После генерации все Jupyter Notebook были отдельно выполнены.

Это позволяет проверить, что автоматически созданные версии способны повторить вычислительные эксперименты без ручных изменений.

```
=====
Выполняется: notebooks/mmc/mmc.ipynb
=====
[NbConvertApp] Converting notebook notebooks/mmc/mmc.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 14320 bytes to notebooks/mmc/mmc.ipynb
ГOTOBO: notebooks/mmc/mmc.ipynb

=====
Выполняется: notebooks/ross/ross.ipynb
=====
[NbConvertApp] Converting notebook notebooks/ross/ross.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 8944 bytes to notebooks/ross/ross.ipynb
ГOTOBO: notebooks/ross/ross.ipynb

=====
Выполняется: notebooks/ross__param/ross__param.ipynb
=====
[NbConvertApp] Converting notebook notebooks/ross__param/ross__param.ipynb t
o notebook
Starting kernel event loops.
[NbConvertApp] Writing 33329 bytes to notebooks/ross__param/ross__param.ipyn
b
ГOTOBO: notebooks/ross__param/ross__param.ipynb
(jupyter) kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2
026-1-study-simulation-modeling/labs/lab07/project$ |
```

Рисунок 9.1: Выполнение Jupyter Notebook

10 Структура проекта

Итоговый проект содержит:

- исходные литературные программы;
- сгенерированный Julia-код;
- Jupyter Notebook;
- Quarto-документацию;
- CSV-файлы результатов;
- оригинальные графики.

```

2026-1-study-simulation-modeling/labs/lab07/project$ deactivate
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ find scripts markdown notebooks data plots \
    -maxdepth 2 \
    -type f \
    | sort
data/mmc_results.csv
data/ross_parameter_summary.csv
markdown/mmc/mmc.qmd
markdown/ross/ross.qmd
markdown/ross__param/ross__param.qmd
notebooks/mmc/mmc.ipynb
notebooks/ross/ross.ipynb
notebooks/ross__param/ross__param.ipynb
plots/mmc_timeline.png
plots/mmc_waiting_time.png
plots/ross_crash_time.png
plots/ross_healthy.png
plots/ross_queue.png
plots/ross_utilization.png
scripts/intro.jl
scripts/mmc.jl
scripts/mmc/mmc.jl
scripts/ross.jl
scripts/ross/ross.jl
scripts/ross__param.jl
scripts/ross__param/ross__param.jl
scripts/tangle.jl
scripts/test_setup.jl

```

Рисунок 10.1: Итоговая структура проекта

11 Контроль версий

После выполнения лабораторной работы изменения были сохранены в Git и отправлены в GitVerse и GitHub.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git log --oneline --decorate --graph -6
* 15e6c90 (HEAD -> develop, origin/develop, github/develop) feat(lab07): add discrete-event simulation models
* 92b057a docs(lab06): add report and presentation
* 0f5dcdcf feat(lab06): add SIR Petri net simulations
* 500e89d docs(lab05): add report and presentation
* d9df236 feat(lab05): add Petri net dining philosophers model
* a032a6b docs(lab04): add report and presentation
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ |
```

Рисунок 11.1: Финальное состояние Git

12 Выводы

В лабораторной работе были реализованы две дискретно-событийные модели.

В М/М/с каждая заявка моделируется отдельным процессом, а несколько каналов обслуживания представлены общим ресурсом ограниченной ёмкости.

В модели Росса моделируются процессы отказа машин, использования резерва и восстановления машин ремонтниками.

Расширение модели позволило исследовать влияние количества машин и ремонтников, а также отслеживать очередь на ремонт и загрузку ремонтного оборудования.

Имитационное среднее время до падения было сопоставлено с аналитическим результатом, рассчитанным программно на основе марковской модели.

Использование литературного программирования, Jupyter Notebook и сохранённых результатов обеспечивает воспроизводимость экспериментов.

Приложение А. Документация модели М/М/с

13 Модель М/М/с

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

Основная дискретно-событийная схема соответствует листингу лабораторной работы.

```
ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions

using DataFrames
using CSV
using Plots
```

13.1 Параметры моделирования

```
rng =  
    StableRNG(  
        123  
    )  
  
num_customers =  
    10
```

13.2 Параметры очереди

```
num_servers =  
    2  
  
mu =  
    1.0 / 2  
  
lam =  
    0.9  
  
arrival_dist =  
    Exponential(  
        1 / lam  
    )  
  
service_dist =  
    Exponential(  
        1 / mu  
    )
```

```
    1 / mu  
  )
```

Для требуемых заданием графиков сохраняем времена событий.

```
arrival_times =  
  fill(  
    NaN,  
    num_customers  
  )  
  
service_start_times =  
  fill(  
    NaN,  
    num_customers  
  )  
  
exit_times =  
  fill(  
    NaN,  
    num_customers  
  )
```

13.3 Поведение заявки

```
@resumable function customer(  
  env::Environment,  
  server::Resource,
```

```

    id::Integer,
    t_a::Float64,
    d_s::Distribution
)

@yield timeout(
    env,
    t_a
)

arrival_times[id] =
    now(env)

println(
    "Customer $id arrived: ",
    now(env)
)

@yield request(
    server
)

service_start_times[id] =
    now(env)

println(
    "Customer $id entered service: ",
    now(env)
)

```

```

    )

    @yield timeout(
        env,
        rand(
            rng,
            d_s
        )
    )

    @yield unlock(
        server
    )

    exit_times[id] =
        now(env)

    println(
        "Customer $id exited service: ",
        now(env)
    )
end

```

13.4 Запуск моделирования

```

function setup_and_run()

    sim =

```

```

Simulation()

server =
    Resource(
        sim,
        num_servers
    )

arrival_time =
    0.0

for i in 1:num_customers

    arrival_time +=
        rand(
            rng,
            arrival_dist
        )

    @process customer(
        sim,
        server,
        i,
        arrival_time,
        service_dist
    )

end

```

```
run(sim)
end

setup_and_run()
```

13.5 Результаты

```
df =
    DataFrame(
        customer=
            1:num_customers,
        arrival=
            arrival_times,
        service_start=
            service_start_times,
        exit=
            exit_times
    )

df.waiting =
    df.service_start -
    df.arrival

df.service =
    df.exit -
    df.service_start
```

```

df.system_time =
    df.exit -
    df.arrival

CSV.write(
    datadir(
        "mmc_results.csv"
    ),
    df
)

println(
    "\nXXXXXXXX XXXX XXXXXXXX: ",
    mean(
        df.waiting
    )
)

println(
    "XXXXXXXX XXXX X XXXXXXXX: ",
    mean(
        df.system_time
    )
)

```

13.6 График событий

```
p1 =  
    plot(  
        df.customer,  
        df.arrival;  
        marker=:circle,  
        label="Arrival",  
        xlabel="Customer",  
        ylabel="Time",  
        title="M/M/c: XXXXXXXX XXXXXXXX"  
    )  
  
plot!(  
    p1,  
    df.customer,  
    df.service_start;  
    marker=:circle,  
    label="Service start"  
)  
  
plot!(  
    p1,  
    df.customer,  
    df.exit;  
    marker=:circle,  
    label="Exit"  
)
```

```

savefig(
    p1,
    plotsdir(
        "mmc_timeline.png"
    )
)

```

13.7 Время ожидания

```

p2 =
    bar(
        df.customer,
        df.waiting;
        xlabel="Customer",
        ylabel="Waiting time",
        title="M/M/c: XXXXX XXXXXXXX",
        legend=false
    )

savefig(
    p2,
    plotsdir(
        "mmc_waiting_time.png"
    )
)

println(

```

```
        "\nXXXXXXXXXX: "  
    )  
  
    println(  
        "data/mmc_results.csv"  
    )  
  
    println(  
        "plots/mmc_timeline.png"  
    )  
  
    println(  
        "plots/mmc_waiting_time.png"  
    )
```

Приложение Б. Документация базовой модели Росса

14 Модель Росса

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

Базовая реализация соответствует листингу лабораторной работы.

```
using DrWatson
@quickactivate "project"

using ResumableFunctions
using ConcurrentSim

using Distributions
using Random
using StableRNGs

const RUNS =
    5

const N =
    10

const S =
```

3

```
const SEED =
```

150

```
const LAMBDA =
```

100

```
const MU =
```

1

```
const rng =
```

```
  StableRNG(
```

42

```
  )
```

```
const F =
```

```
  Exponential(
```

LAMBDA

```
  )
```

```
const G =
```

```
  Exponential(
```

MU

```
  )
```

14.1 Поведение машины

```
@resumable function machine(  
  env::Environment,  
  repair_facility::Resource,  
  spares::Store{Process}  
)  
  
  while true  
  
    try  
  
      @yield timeout(  
        env,  
        Inf  
      )  
  
    catch  
    end  
  
    @yield timeout(  
      env,  
      rand(  
        rng,  
        F  
      )  
    )  
  )  
end
```

```

get_spare =
    take!(
        spares
    )

@yield get_spare |
    timeout(env)

if state(
    get_spare
) != ConcurrentSim.idle

    @yield interrupt(
        value(
            get_spare
        )
    )

else

    throw(
        StopSimulation(
            "No more spares!"
        )
    )

end

@yield request(

```

```

        repair_facility
    )

    @yield timeout(
        env,
        rand(
            rng,
            G
        )
    )

    @yield unlock(
        repair_facility
    )

    @yield put!(
        spares,
        active_process(
            env
        )
    )
end
end

```

14.2 Начальное состояние

```

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process}
)

    for i in 1:N

        proc =
            @process machine(
                env,
                repair_facility,
                spares
            )

            @yield interrupt(
                proc
            )
    end

    for i in 1:S

        proc =
            @process machine(
                env,
                repair_facility,
                spares
            )
    end

```

```

        @yield put!(
            spares,
            proc
        )
    end
end
end

```

14.3 Один прогон

```

function sim_repair()

    sim =
        Simulation()

    repair_facility =
        Resource(
            sim
        )

    spares =
        Store{Process}(
            sim
        )

    @process start_sim(
        sim,
        repair_facility,

```

```

        spares
    )

    msg =
        run(sim)

    stop_time =
        now(sim)

    println(
        "At time $stop_time: $msg"
    )

    stop_time
end

```

14.4 Несколько повторов

```

results =
    Float64[]

for i in 1:RUNS

    push!(
        results,
        sim_repair()
    )
end

```

```
println(  
    "Average crash time: ",  
    sum(results) /  
    RUNS  
)
```

Приложение В. Документация параметрической модели Росса

15 Расширение модели Росса

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

Реализуются дополнительные пункты задания: несколько ремонтников, разные N, мониторинг очереди и загрузки, график числа исправных машин и аналитическое сравнение.

```
ENV["GKSvstype"] = "100"
```

```
using DrWatson
```

```
@quickactivate "project"
```

```
using ResumableFunctions
```

```
using ConcurrentSim
```

```
using Distributions
```

```
using StableRNGs
```

```
using DataFrames
```

```
using CSV
```

```
using Plots
```

```
using Statistics
```

```
using LinearAlgebra
```

15.1 Параметры из исходной модели

```
const RUNS_PARAM =  
    5  
  
const S_PARAM =  
    3  
  
const LAMBDA_PARAM =  
    100.0  
  
const MU_PARAM =  
    1.0  
  
const SEED_PARAM =  
    150
```

15.2 Мониторинг

```
mutable struct RepairMonitor  
  
    times::Vector{Float64}  
  
    healthy::Vector{Int}  
  
    queue::Vector{Int}  
  
    busy::Vector{Int}
```

```

    healthy_now::Int

    queue_now::Int

    busy_now::Int
end

function RepairMonitor(
    total_machines::Int
)

    return RepairMonitor(
        [0.0],
        [total_machines],
        [0],
        [0],
        total_machines,
        0,
        0
    )
end

function record!(
    monitor::RepairMonitor,
    env::Environment
)

```

```
push!(  
    monitor.times,  
    now(env)  
)  
  
push!(  
    monitor.healthy,  
    monitor.healthy_now  
)  
  
push!(  
    monitor.queue,  
    monitor.queue_now  
)  
  
push!(  
    monitor.busy,  
    monitor.busy_now  
)  
  
return nothing  
end
```

15.3 Поведение машины

```

@resumable function machine_param(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    rng,
    F,
    G,
    monitor::RepairMonitor
)

    while true

        try

            @yield timeout(
                env,
                Inf
            )

        catch
        end

        @yield timeout(
            env,
            rand(
                rng,
                F
            )

```

```

)

monitor.healthy_now -=
    1

record!(
    monitor,
    env
)

get_spare =
    take!(
        spares
    )

@yield get_spare |
    timeout(env)

if state(
    get_spare
) != ConcurrentSim.idle

    @yield interrupt(
        value(
            get_spare
        )
    )
)

```

```

else

    throw(
        StopSimulation(
            "No more spares!"
        )
    )
end

monitor.queue_now +=
    1

record!(
    monitor,
    env
)

@yield request(
    repair_facility
)

monitor.queue_now -=
    1

monitor.busy_now +=
    1

record!(

```

```

        monitor,
        env
    )

    @yield timeout(
        env,
        rand(
            rng,
            G
        )
    )

    @yield unlock(
        repair_facility
    )

    monitor.busy_now -=
        1

    monitor.healthy_now +=
        1

    record!(
        monitor,
        env
    )

    @yield put!(

```

```

        spares,
        active_process(
            env
        )
    )
end
end

```

15.4 Начальное состояние

```

@resumable function start_sim_param(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    N::Int,
    S::Int,
    rng,
    F,
    G,
    monitor::RepairMonitor
)

    for i in 1:N

        proc =

            @process machine_param(
                env,
                repair_facility,

```

```

        spares,
        rng,
        F,
        G,
        monitor
    )

    @yield interrupt(
        proc
    )
end

for i in 1:S

    proc =
        @process machine_param(
            env,
            repair_facility,
            spares,
            rng,
            F,
            G,
            monitor
        )

        @yield put!(
            spares,
            proc

```

```
)  
end  
end
```

15.5 Один параметрический прогон

```
function sim_repair_param(  
    N::Int,  
    S::Int,  
    num_repairers::Int,  
    seed::Int  
)  
  
    rng =  
        StableRNG(  
            seed  
        )  
  
    F =  
        Exponential(  
            LAMBDA_PARAM  
        )  
  
    G =  
        Exponential(  
            MU_PARAM  
        )
```

```

sim =
    Simulation()

repair_facility =
    Resource(
        sim,
        num_repairers
    )

spares =
    Store{Process}(
        sim
    )

monitor =
    RepairMonitor(
        N + S
    )

@process start_sim_param(
    sim,
    repair_facility,
    spares,
    N,
    S,
    rng,
    F,
    G,

```

```

        monitor
    )

    msg =
        run(sim)

    stop_time =
        now(sim)

    return (
        stop_time,
        msg,
        monitor
    )
end

```

15.6 Среднее значение показателя во времени

```

function time_average(
    times,
    values,
    stop_time
)

    total =
        0.0

    for i in 1:(

```

```

        length(times) - 1
    )

    total +=
        values[i] *
        (
            times[i + 1] -
            times[i]
        )
end

if !isempty(times)

    total +=
        values[end] *
        (
            stop_time -
            times[end]
        )
end

return total /
    stop_time
end

```

15.7 Аналитическое среднее время до падения

Состояние — число исправных машин. Переходы соответствуют описанной в методичке марковской модели.

```
function analytic_crash_time(
    N::Int,
    S::Int,
    num_repairers::Int
)

    states =
        collect(
            N: (N + S)
        )

    count_states =
        length(
            states
        )

    Q =
        zeros(
            Float64,
            count_states,
            count_states
        )

    for (
```

```

        row,
        healthy
    ) in enumerate(
        states
    )

    failure_rate =
        min(
            N,
            healthy
        ) /
        LAMBDA_PARAM

    broken =
        N +
        S -
        healthy

    repair_rate =
        min(
            num_repairers,
            broken
        ) /
        MU_PARAM

    Q[
        row,
        row

```

```

] =
    -(
        failure_rate +
        repair_rate
    )

if healthy > N

    Q[
        row,
        row - 1
    ] =
        failure_rate
end

if healthy < N + S

    Q[
        row,
        row + 1
    ] =
        repair_rate
end
end

expected_times =
    -(
        Q \

```

```

        ones(
            count_states
        )
    )

    return expected_times[end]
end

```

15.8 Набор параметров

```

N_values =
    [
        5,
        10,
        15
    ]

repairer_values =
    [
        1,
        2
    ]

results =
    NamedTuple[]

representative_monitor =
    nothing

```

```
representative_stop_time =  
    0.0
```

15.9 Серия экспериментов

```
for N in N_values  
  
    for num_repairers in repairer_values  
  
        crash_times =  
            Float64[]  
  
        queues =  
            Float64[]  
  
        utilizations =  
            Float64[]  
  
        for run_id in 1:RUNS_PARAM  
  
            seed =  
                SEED_PARAM +  
                100 * N +  
                10 * num_repairers +  
                run_id  
  
            stop_time,
```

```

msg,
monitor =
    sim_repair_param(
        N,
        S_PARAM,
        num_repairers,
        seed
    )

avg_queue =
    time_average(
        monitor.times,
        monitor.queue,
        stop_time
    )

avg_busy =
    time_average(
        monitor.times,
        monitor.busy,
        stop_time
    )

utilization =
    avg_busy /
    num_repairers

push! (

```

```

        crash_times,
        stop_time
    )

    push!(
        queues,
        avg_queue
    )

    push!(
        utilizations,
        utilization
    )

    if N == 10 &&
        num_repairers == 2 &&
        run_id == 1

        global representative_monitor =
            monitor

        global representative_stop_time =
            stop_time
    end
end

analytic_time =
    analytic_crash_time(

```

```

        N,
        S_PARAM,
        num_repairers
    )

simulated_time =
    mean(
        crash_times
    )

push!(
    results,
    (
        N=N,
        S=S_PARAM,
        repairers=
            num_repairers,
        mean_crash_time=
            simulated_time,
        analytic_crash_time=
            analytic_time,
        relative_error=
            abs(
                simulated_time -
                analytic_time
            ) /
            analytic_time,
        average_queue=

```

```

        mean(
            queues
        ),
        repair_utilization=
            mean(
                utilizations
            )
    )
)

println(
    "N=",
    N,
    ", repairers=",
    num_repairers,
    " | simulation=",
    round(
        simulated_time,
        digits=3
    ),
    " | analytic=",
    round(
        analytic_time,
        digits=3
    ),
    " | queue=",
    round(
        mean(

```

```

        queues
    ),
    digits=4
),
" | utilization=",
round(
    mean(
        utilizations
    ),
    digits=4
)
)
end
end

```

15.10 Таблица результатов

```

df_summary =
    DataFrame(
        results
    )

CSV.write(
    datadir(
        "ross_parameter_summary.csv"
    ),
    df_summary
)

```



```

    p_healthy,
    plotsdir(
        "ross_healthy.png"
    )
)

```

15.12 График очереди на ремонт

```

p_queue =
    plot(
        representative_monitor.times,
        representative_monitor.queue;
        xlabel="time",
        ylabel="queue length",
        title=
            "Queue length vs time",
        label="Queue",
        linewidth=2
    )

savefig(
    p_queue,
    plotsdir(
        "ross_queue.png"
    )
)

```

15.13 Загрузка ремонтников

```
p_utilization =  
    plot(  
        xlabel="N",  
        ylabel="Utilization",  
        title=  
            "XXXXXXXXXX XXXXXXXXXXXXXXXX"  
    )  
  
for repairers in repairer_values  
  
    part =  
        filter(  
            :repairers =>  
                value ->  
                    value == repairers,  
            df_summary  
        )  
  
    plot!(  
        p_utilization,  
        part.N,  
        part.repair_utilization;  
        marker=:circle,  
        linewidth=2,  
        label=  
            "$repairers repairer(s)"
```

```

    )
end

savefig(
    p_utilization,
    plotsdir(
        "ross_utilization.png"
    )
)

```

15.14 Имитационное и аналитическое время падения

```

p_crash =
    plot(
        xlabel="N",
        ylabel="XXXXXXXX XXXX XX XXXXXXX",
        title=
            "XXXXXXXXXXXXX X XXXXXXXXXXXXXXX XXXXXXX"
    )

for repairers in repairer_values

    part =
        filter(
            :repairers =>
                value ->

```

```

        value == repairers,
        df_summary
    )

    plot!(
        p_crash,
        part.N,
        part.mean_crash_time;
        marker=:circle,
        linewidth=2,
        label=
            "Simulation, r=$repairers"
    )

    plot!(
        p_crash,
        part.N,
        part.analytic_crash_time;
        marker=:square,
        linewidth=2,
        linestyle=:dash,
        label=
            "Analytic, r=$repairers"
    )
end

savefig(
    p_crash,

```

```

        plotsdir(
            "ross_crash_time.png"
        )
    )

println(
    "\nXXXXXXXXXX: "
)

println(
    "data/ross_parameter_summary.csv"
)

println(
    "plots/ross_healthy.png"
)

println(
    "plots/ross_queue.png"
)

println(
    "plots/ross_utilization.png"
)

println(
    "plots/ross_crash_time.png"
)

```

)